

2303AS010-NATURAL LANGUAGE PROCESSING

MODULE-4

Annotating Linguistic Structure

1. Discuss how sequence labeling is applied to perform both Part-of-Speech (POS) tagging and Named Entity Recognition (NER) for the sentence:

"Barack Obama was born in Hawaii."

Illustrate the output of the labeling process, detailing the tags used for both tasks.

Sequence Labeling

Sequence labeling is a process in Natural Language Processing where each word in a sentence is assigned a label based on its role.

Input: Sentence

Output: Labels for each word

Example sentence:

"Barack Obama was born in Hawaii."

Step 1: Tokenization

Split the sentence into individual words (tokens).

Output:

[Barack, Obama, was, born, in, Hawaii, .]

Step 2: Feature Identification

For each word, the system checks:

- Capital letters (e.g., Barack → likely a name)
- Word position
- Surrounding words (context)
- Suffix/prefix (e.g., "-ing", "-ed")

Example:

- "Barack" → Starts with capital → possible proper noun
- "in" → Common preposition

Step 3: POS Tagging (Grammatical Labeling)

Each word is assigned a Part-of-Speech tag.

Word	POS Tag	Description
Barack	NNP	Proper noun, singular
Obama	NNP	Proper noun, singular
was	VBD	Verb, past tense
born	VCN	Verb, past participle
in	IN	Preposition
Hawaii	NNP	Proper noun, singular
.	.	Punctuation

Step 4: NER Tagging (Entity Identification)

Now identify **real-world entities** like person, place, etc.

Tagging Format (BIO Method)

- **B** → Beginning
- **I** → Inside
- **O** → Outside

NER identifies proper names and classifies them into predefined categories such as **Person (PER)**, **Location (LOC)**, **Organization (ORG)**, etc. Using the **IOB (Inside-Outside-Beginning) tagging scheme**, the NER-labeled output is,

Word	NER Tag	Description
Barack	B-PER	Beginning of a Person name
Obama	I-PER	Inside a Person name
was	O	Outside any named entity
born	O	Outside any named entity
in	O	Outside any named entity
Hawaii	B-LOC	Beginning of a Location
.	O	Outside any named entity

Step 5: Model Processing

Different models are used to assign labels:

- Rule-based methods
- Hidden Markov Model (HMM)
- Conditional Random Fields (CRF)
- Deep Learning (BiLSTM, Transformers)

These models use probability and context to predict correct tags.

1. Rule-Based Methods

Definition

Rule-based systems assign tags using **manually written linguistic rules**.

How it works

- Uses grammar rules and dictionaries
- Example rules:
 - If a word starts with a capital letter → Proper Noun
 - If a word ends with “-ing” → Verb

Example

"Barack" → Capitalized → Proper noun (NNP)

2. Hidden Markov Model (HMM)

Definition

HMM is a **probabilistic model** that predicts tags based on:

- Previous tag
- Current word

Key Idea

It uses probability to find the **most likely sequence of tags**.

How it works

- Calculates:
 - **Transition Probability** → Probability of one tag following another
 - **Emission Probability** → Probability of a word given a tag

Example:

- Probability that a **verb follows a noun**
- Probability that the word "born" is a verb

3. Conditional Random Fields (CRF)

Definition

CRF is an advanced probabilistic model used for **sequence labeling**.

Key Idea

It considers the **entire sentence context**, not just previous words.

How it works

- Looks at:
 - Current word
 - Previous word
 - Next word
- Predicts tags jointly for the whole sequence

Example:

- "Barack Obama" → CRF recognizes both words together as a person

4. Deep Learning Models (BiLSTM, Transformers)

Definition

Modern models that use **neural networks** to learn patterns automatically.

a) BiLSTM (Bidirectional LSTM)

How it works

- Reads the sentence in:
 - Forward direction → left to right
 - Backward direction → right to left

Example:

- Understands context before and after a word

b) Transformers

How it works

- Uses **attention mechanism**
- Looks at all words in the sentence at once

Example:

- "Obama" connects with "Barack" even if far apart

2. Apply a rule-based and statistical approach to tag the parts of speech in the sentence:

"The quick brown fox jumps over the lazy dog."

Compare and contrast the two methods, highlighting their strengths and weaknesses.

Part-of-Speech (POS) tagging is an important task in Natural Language Processing where each word in a sentence is assigned a grammatical label such as noun, verb, adjective, etc.

Sentence:

"The quick brown fox jumps over the lazy dog."

Tokenization

Split the sentence into words:

[The, quick, brown, fox, jumps, over, the, lazy, dog, .]

Rule-Based POS Tagging

Definition

Rule-based tagging assigns POS tags using **predefined grammar rules and dictionaries.**

Rules Applied

- "The" → Determiner (DT)
- Words before a noun describing it → Adjective (JJ)
- "fox", "dog" → Noun (NN)
- Verb form ending with "s" → Verb (VBZ)
- "over" → Preposition (IN)

Output (Rule-Based)

Word	POS Tag	Reason
The	DT	Determiner
quick	JJ	Describes noun
brown	JJ	Describes noun
fox	NN	Noun
jumps	VBZ	Verb (3rd person)
over	IN	Preposition
the	DT	Determiner
lazy	JJ	Describes noun
dog	NN	Noun
.	.	Punctuation

Statistical Approach (HMM Example)

Definition

Statistical models like Hidden Markov Model assign tags based on **probability learned from data**.

How It Works

- Uses:
 - Transition probability $\rightarrow P(\text{Tag}_n \mid \text{Tag}_{n-1})$
 - Emission probability $\rightarrow P(\text{Word} \mid \text{Tag})$
- Chooses the most probable tag sequence

Output (Statistical Model)

Word	POS
The	DT
quick	JJ
brown	JJ
fox	NN
jumps	VBZ
over	IN
the	DT
lazy	JJ
dog	NN

Comparison of Rule-Based vs Statistical Methods

Feature	Rule-Based	Statistical (e.g., HMM, CRF)
Basis	Hand-crafted grammar rules	Probability from labeled data
Adaptability	Poor for ambiguous/unseen structures	Learns from data; adapts better

Performance on Simple Sentences	Very accurate	Also accurate
Performance on Complex/Ambiguous Sentences	May fail due to missing rules	Handles ambiguity well using context
Speed	Fast (no training)	Requires training, but fast inference
Explainability	Transparent rules	Black-box for non-linear models
Maintenance	Rules must be updated manually	Automatically updated with more data

Strengths and Weaknesses

Rule-Based Approach

Strengths

- Easy to understand
- No training data required
- Works well for simple sentences

Weaknesses

- Cannot handle ambiguity
- Requires many rules
- Not scalable

Statistical Approach (HMM)

Strengths

- Handles ambiguity better
- Learns from data
- More adaptable

Weaknesses

- Needs large training data
- Computationally complex
- Assumes independence between words

Conclusion

Both approaches aim to assign correct POS tags:

- **Rule-based methods** are simple but limited
- **Statistical models like HMM** are more powerful and flexible

In modern systems, statistical and deep learning methods are preferred for better accuracy in real-world applications.

3. Explore syntactic ambiguity by generating two different parse trees for the sentence:

"The man saw the boy with a telescope."

Describe how ambiguity arises and discuss how it affects natural language understanding.

Syntactic ambiguity occurs when a sentence can be parsed in more than one way, leading to different meanings. This is an important concept in Natural Language Processing.

Syntactic ambiguity by generating two different parse trees for the sentence:
"The man saw the boy with a telescope."

"The man saw the boy with a telescope."

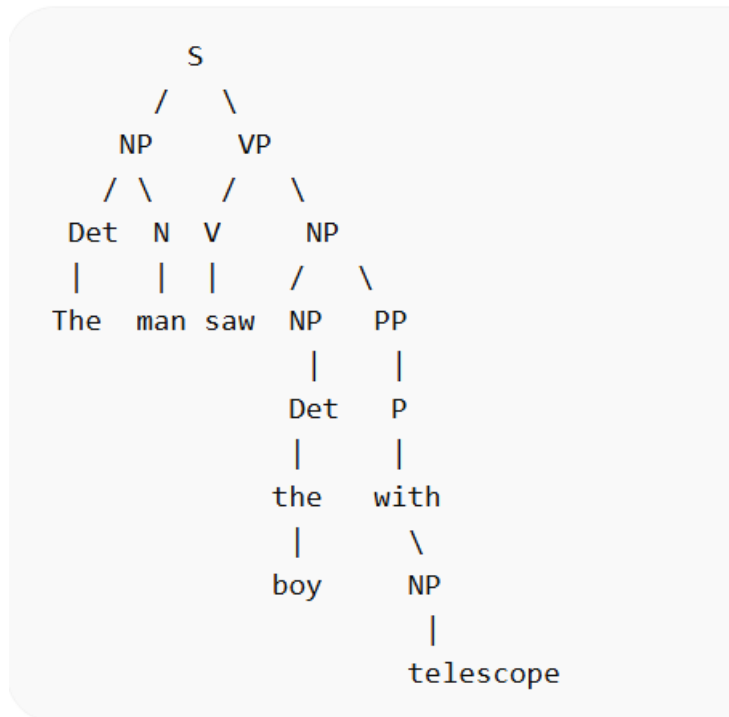
At first glance, it's unclear **who** has the telescope:

Is it the man who used the telescope to see the boy?

Or is it the boy who has the telescope?

This is **syntactic ambiguity** — **the structure** of the sentence allows for **multiple interpretations**.

Parse 1: (*The man used a telescope to see the boy*)



Step 1: Start from the top (S)

- S = Sentence
- A sentence is divided into:
 - NP (Noun Phrase) → Subject
 - VP (Verb Phrase) → Action + Object

Step 2: Subject (NP)

NP = “The man”

- Det (Determiner) → *The*
- N (Noun) → *man*

Step 3: Verb Phrase (VP)

VP = “saw the boy with a telescope”

- V → *saw* (action)
- NP → object
- PP → extra information

Step 4: Object (NP inside VP)

NP = “the boy”

This is the **object of the verb “saw”**

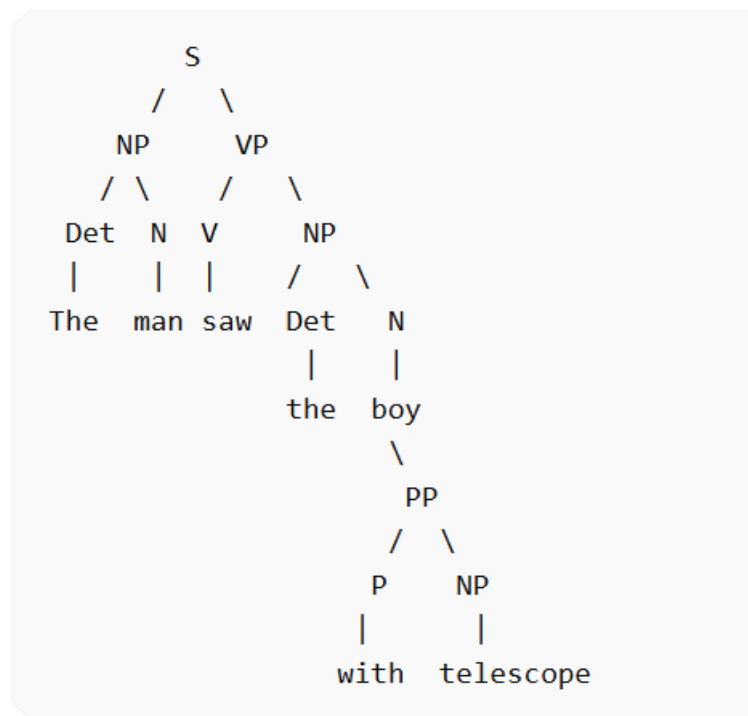
Step 5: Prepositional Phrase (PP)

PP = “with a telescope”

- P (Preposition) → *with*
- NP → *telescope*

“The man saw the boy using a telescope”

Parse Tree 2 (Boy has the telescope)



Ambiguity increases complexity for both humans and machines trying to understand language.

For humans:

Usually, context or world knowledge helps resolve ambiguity.

(e.g., you know that telescopes are usually used to look, not carried by boys randomly.)

For NLP systems (like ChatGPT, search engines, translation tools):

How Ambiguity Arises

- The phrase “**with a telescope**” is a **Prepositional Phrase (PP)**
- It can attach to:
 - **Verb Phrase (VP)** → action meaning
 - **Noun Phrase (NP)** → object description

This multiple attachment leads to **syntactic ambiguity**

Effects on Natural Language Understanding

1. Meaning Confusion

- Same sentence → multiple meanings
- Difficult for machines to interpret correctly

2. Impact on NLP Applications

- Machine Translation
- Chatbots
- Information Retrieval

Incorrect parsing leads to **wrong outputs**

3. Need for Disambiguation

To resolve ambiguity, systems use:

- **Probabilistic methods** → choose the most likely meaning
- **Machine Learning** → learn from data and predict
- **Semantic analysis** → use context and real-world meaning

4. Create a context-free grammar (CFG) for the sentence:

"The cat sat on the mat."

Perform constituency parsing on the sentence and illustrate the resulting parse tree.

A Context-Free Grammar (CFG) is a formal system used to describe the syntactic structure of languages, especially natural language. It consists of:

Non-terminals

Non-terminals are **syntactic variables or symbols** used in a grammar to represent groups of words or phrases. They help in **forming the structure of a sentence** and can be further expanded using production rules.

Examples: categories like sentence (S), noun phrase (NP), verb phrase (VP).

Terminals

Terminals are the **actual words or symbols** in a language that appear in the final sentence. They **cannot be expanded further** in the grammar.

Examples: the, cat, sat, on

Rules: show how non-terminals can be broken into smaller parts (e.g., $S \rightarrow NP VP$).

Start symbol: usually S, the whole sentence.

Definition

A CFG is a 4-tuple $G = (N, \Sigma, R, S)$:

- **N**: a finite set of non-terminal symbols (e.g., S, NP, VP)
- **Σ** : a finite set of terminal symbols (words like the, cat, sat, on)
- **R**: a set of production rules of the form $A \rightarrow \beta$, where:
 - A is a non-terminal
 - β is a string of terminals and/or non-terminals
- **S**: the designated start symbol (typically S for sentence)

Purpose

- **Generative model**: CFGs *generate* all and only the valid sentences of a language.
- **Structural model**: CFGs describe *constituent structure*, i.e., how words group into phrases.
- The **basic sentence rule** in CFG:
 - $S \rightarrow NP VP$
- a **Sentence (S)** consists of a **Noun Phrase (NP)** followed by a **Verb Phrase (VP)**.

Context-Free Grammar (CFG) for the sentence:

"The cat sat on the mat. "

Lexical categories:

"The" – Determiner (Det)

"cat", "mat" – Nouns (Noun)

"sat" – Verb (Verb)

"on" – Preposition (Prep)

Non-terminal rules:

$S \rightarrow NP VP$

$NP \rightarrow Det Noun$

$VP \rightarrow Verb$

$VP \rightarrow Verb PP$

$PP \rightarrow Prep NP$

Lexicon rules (terminal rules):

$Det \rightarrow the$

$Noun \rightarrow cat \mid mat$

$Verb \rightarrow sat$

$Prep \rightarrow on$

Constituency Parsing and Parse Tree

Constituency parsing is a method of breaking a sentence into phrases and showing their grammatical structure in a tree form.

Step-by-step derivation:

$S \rightarrow NP VP$

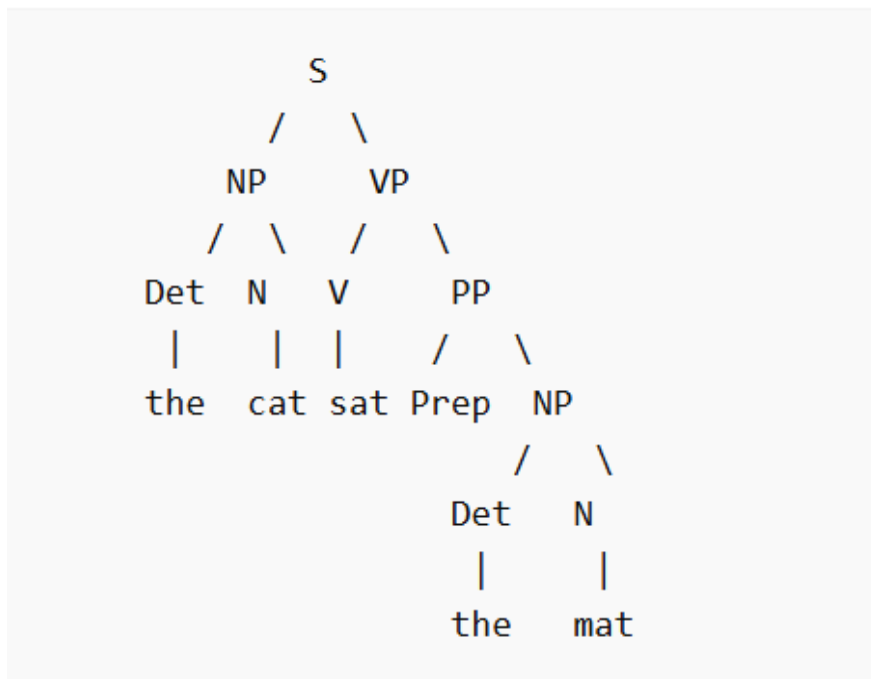
$NP \rightarrow Det Noun \rightarrow the cat$

VP → Verb PP

Verb → sat

PP → Prep NP → on the mat

NP → Det Noun → the mat



5. Evaluate the performance of an NER model using the following data:

True entities: [Barack Obama, Hawaii, Google]

Predicted entities: [Obama, Hawaii, Google LLC]

Calculate precision, recall, and F1-score and interpret the results.

NER model

- An NER model (Named Entity Recognition model) is a type of Natural Language Processing (NLP) model that identifies and classifies important words or phrases in a sentence into predefined categories.

- An NER model finds named entities in text and labels them.
- Named entities are real-world objects such as:
- Person names → “Barack Obama”
- Places → “Hawaii”
- Organizations → “Google”
- Dates → “April 13, 2026”

True entities: [Barack Obama, Hawaii, Google]

Predicted entities: [Obama, Hawaii, Google LLC]

Calculate precision, recall, and F1-score and interpret the results.

Step 1: Identify Matches

True entities: Barack Obama, Hawaii, Google

Predicted entities: Obama, Hawaii, Google LLC

Now check exact matches:

Hawaii = Hawaii (Correct)

Barack Obama ≠ Obama (Partial, but counted as wrong in strict evaluation)

Google ≠ Google LLC (Different entity, counted as wrong)

So:

True Positives (TP) = 1 (Hawaii)

False Positives (FP) = 2 (Obama, Google LLC)

False Negatives (FN) = 2 (Barack Obama, Google)

- TP: Correctly predicted positive
- TN: Correctly predicted negative
- FP: Incorrectly predicted positive
- FN: Missed positive cases

Actual	Predicted	Type
1	1	TP
0	0	TN
0	1	FP
1	0	FN

1. Precision

- **Precision is the ratio of correctly predicted positive observations to the total predicted positives.**

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Explanation:**
- It tells **how accurate the model's predictions are**
- Out of all predicted entities, **how many are actually correct**

Example:

- If model predicts 10 entities and 7 are correct → Precision = 7/10 = 0.7

2. Recall

- **Recall is the ratio of correctly predicted positive observations to all actual positives.**

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

•

Explanation:

- It tells **how well the model finds all real entities**
- Out of all true entities, **how many the model detected**

Example:

- If there are 10 real entities and model finds 6 → Recall = 6/10 = 0.6

3. F1-score

- **F1-score is the harmonic mean of Precision and Recall.**

Formula:

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Explanation:

- It gives a **balance between Precision and Recall**
- Useful when both are important
- *“Overall performance of the model”*

Step 2: Calculate Metrics

Precision

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{1}{1 + 2} = \frac{1}{3} = 0.33$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{1}{1 + 2} = \frac{1}{3} = 0.33$$

F1-Score

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times 0.33 \times 0.33}{0.33 + 0.33} = 0.33$$

Final Answer

- Precision = 0.33
- Recall = 0.33

- F1-score = 0.33

Interpretation

- The model has **low performance**:
- **Low precision (0.33)** → Many predicted entities are incorrect.
- **Low recall (0.33)** → The model fails to detect many true entities.
- The model struggles with:
 - **Entity boundaries** (e.g., "Barack Obama" vs "Obama")
 - **Entity variations** (e.g., "Google" vs "Google LLC")

6. Use the CKY parsing algorithm to parse the sentence:

"John likes Mary."

Assume the following grammar rules:

$S \rightarrow NP VP$

$VP \rightarrow V NP$

$NP \rightarrow \text{John} \mid \text{Mary}$

$V \rightarrow \text{likes}$

Demonstrate step-by-step how the algorithm constructs the parse.

CKY parsing (Cocke–Kasami–Younger Algorithm)

CKY parsing is a bottom-up parsing algorithm used to determine whether a sentence can be generated by a context-free grammar (CFG) in Chomsky Normal Form (CNF).

Chomsky Normal Form (CNF) is a simplified form of a context-free grammar (CFG) in which every production rule follows a specific structure.

Rules in CNF:

A grammar is in CNF if all productions are of the form:

1. $A \rightarrow BC$
 - A, B, C are non-terminals
 - (two non-terminals on the right side)
2. $A \rightarrow a$
 - A is a non-terminal

- a is a terminal (word)

Step 1: Given Grammar

Convert rules into CNF

- $S \rightarrow NP VP$
- $VP \rightarrow V NP$
- $NP \rightarrow John \mid Mary$
- $V \rightarrow likes$

Step 2: Sentence Tokens

- Split the sentence:

Position	Word
1	John
2	likes
3	Mary

Step 3: Initialize CKY Table

We create a **triangular table**.

Fill diagonal (length = 1 substrings)

Check which rules generate each word:

- **John** $\rightarrow$ NP
- **likes** $\rightarrow$ V
- **Mary** $\rightarrow$ NP

(i): Fill Diagonal (Single Words)

i\j	1	2	3
1	NP		
2		V	
3			NP

(ii) Length = 2 Substrings

Span (1,2): “John likes”

Check combinations:

- NP + V $\rightarrow$ No rule

Result: **Empty ($\emptyset$)**

Span (2,3): “likes Mary”

Check:

- $V + NP \rightarrow VP$

Fill:

- $(2,3) = VP$

i\j	1	2	3
1	NP	$\emptyset$	
2		V	VP
3			NP

(iii) Length = 3 (Full Sentence)

Span (1,3): “John likes Mary”

Try all possible splits:

Split at $k = 1$:

$(1,1) = NP$

$(2,3) = VP$

Check:

$NP + VP \rightarrow S$

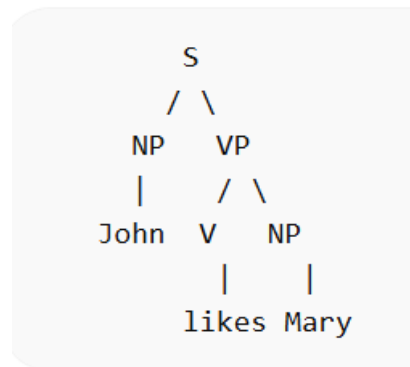
Fill:

$(1,3) = S$

Final CKY Table

i\j	1	2	3
1	NP	$\emptyset$	S
2		V	VP
3			NP

Parse Tree Construction



- CKY is a **bottom-up parsing algorithm**
- It uses **dynamic programming**
- **words** → **phrases** → **sentence**
- It checks all possible splits and rules

7.Solve the dependency parsing problem for the sentence:

"He eats an apple."

Demonstrate the step-by-step transitions (shift, reduce, left-arc, right-arc) used to build the dependency structure.

Dependency Parsing

Dependency Parsing is a syntactic analysis method that focuses on the grammatical relationships between words in a sentence. It represents the structure of a sentence by linking words based on head–dependent relationships, forming a dependency tree.

Dependency Grammar

Models syntax by linking words with binary, asymmetrical relations (head → dependent).

Each word (except the root) depends on another word (its head).

The collection of dependencies forms a **dependency tree**.

Root is typically the main verb of the sentence.

Dependency Tree Properties

Single-head: Every word has exactly one head.

Acyclic: No cycles in the tree.

Connected: All words are connected to the root.

Dependency Parsing Approaches

Transition-Based Parsing

Builds parse incrementally using transitions between parser states.

Common algorithms: **Arc-standard**, **Arc-eager**.

Uses a stack, buffer, and set of dependency arcs.

Often trained using **machine learning** or neural networks.

Graph-Based Parsing

Treats parsing as finding the highest-scoring tree over all possible trees.

Uses algorithms like **Chu-Liu/Edmonds** for non-projective parsing.

Relies on scoring functions over possible arcs (often learned using supervised methods).

Neural Dependency Parsing

Use embeddings and deep learning (e.g., BiLSTM + MLP classifiers).

Encode context better, leading to improved accuracy.

Arc-Standard Parser Basics

The parser uses three elements:

Stack: Holds partially processed words.

Buffer: Holds remaining input words.

Arcs: Stores dependency relations (head $\rightarrow$ dependent).

Operations:

SHIFT: Move the first word from the buffer to the stack.

LEFT-ARC(label): Add an arc from top of the stack to the second-top

(stack[-1] $\leftarrow$ stack[-2]), then remove the second-top.

RIGHT-ARC(label): Add an arc from the second-top of the stack to the top

(stack[-2] $\rightarrow$ stack[-1]), then remove the top.

Example Sentence:

"He eats an apple."

Words and POS Tags

Word	POS
He	PRON
eats	VERB
an	DET
apple	NOUN
.	PUNCT

Step-by-Step Parsing (Arc-Standard)

Initial State:

- Stack: [ROOT]
- Buffer: [He, eats, an, apple, .]
- Arcs: $\emptyset$

Step 1: SHIFT

- Stack: [ROOT, He]
- Buffer: [eats, an, apple, .]

Step 2: SHIFT

- Stack: [ROOT, He, eats]
- Buffer: [an, apple, .]

Step 3: LEFT-ARC(nsubj)

- Add arc: eats → He
- Stack: [ROOT, eats]

Arcs: {(eats, He, nsubj)}

Step 4: SHIFT

- Stack: [ROOT, eats, an]
- Buffer: [apple, .]

Step 5: SHIFT

- Stack: [ROOT, eats, an, apple]
- Buffer: [.]

Step 6: LEFT-ARC(det)

Add arc: apple → an

Stack: [ROOT, eats, apple]

Arcs: + (apple, an, det)

Step 7: RIGHT-ARC(dobj)

Add arc: eats → apple

Stack: [ROOT, eats]

Arcs: + (eats, apple, dobj)

Step 8: SHIFT

- Stack: [ROOT, eats, .]
- Buffer: []

Step 8: SHIFT

- Stack: [ROOT, eats, .]
- Buffer: []

Step 9: RIGHT-ARC(punct)

- Add arc: eats $\rightarrow$.
- Stack: [ROOT, eats]
- Arcs: + (eats, ., punct)

Step 10: RIGHT-ARC(root)

- Add arc: ROOT $\rightarrow$ eats
- Stack: [ROOT]
- Parsing complete!

Final Dependency Tree

Head	Dependent	Relation
eats	He	nsubj
apple	an	det
eats	apple	dobj
eats	.	punct
ROOT	eats	root

8. Model a graph-based dependency parsing approach for the sentence:

"The boy plays soccer."

Illustrate how arcs and weights are used to determine dependencies.

Graph-Based Dependency Parsing

Graph-based dependency parsing formulates parsing as a **graph optimization problem**. The goal is to select the **highest-scoring dependency tree** (i.e., the best set of head–dependent arcs) for a sentence from all possible trees.

Each possible dependency arc (head $\rightarrow$ dependent) is assigned a **score**, and the parser chooses the tree that **maximizes the total score** of its arcs.

Core Components

Input Sentence:

A sequence of words: $w_0(\text{ROOT}), w_1, w_2, \dots, w_n$

Complete Directed Graph:

Nodes = words in the sentence

Edges = possible head $\rightarrow$ dependent relations

Each edge is scored (by a model)

Parsing Goal:

Find a tree (subset of arcs) such that:

It's a valid **dependency tree** (connected, acyclic, one head per node)

Has the **highest total score**

Scoring Function

A **scoring function** $s(h,d)$ assigns a score to a possible arc from head h to dependent d .

Modern models use neural networks (like BiLSTMs or Transformers) to compute these scores using contextual word representations.

Algorithms to Find the Best Tree**Projective Parsing:**

Use **Eisner's algorithm** (dynamic programming)

Guarantees projectivity (no crossing arcs)

Non-Projective Parsing:

Use **Chu-Liu/Edmonds' algorithm** (also called MST algorithm)

Allows for non-projective trees (used for free word-order languages)

Advantages of Graph-Based Parsing

Global Optimization: Considers all arcs at once rather than step-by-step.

Accuracy: Tends to be more accurate than greedy parsers.

Flexibility: Can be extended to non-projective parsing.

Sentence:

"The boy plays soccer."

Step-by-Step: Graph-Based Dependency Parsing

1. Create nodes for each word (including ROOT).
2. Construct a complete **directed graph** with all possible head-dependent arcs.
3. Assign **scores (weights)** to arcs.
4. Use a **maximum spanning tree (MST)** algorithm to select the best dependency tree.

Nodes in the Graph

Each word (plus ROOT) becomes a node:

ROOT, The, boy, plays, soccer, .

Biaffine Scoring Function

For each possible head–dependent pair (i, j), compute:

$$\text{score}(i \rightarrow j) = h_i^T * U * d_j + W(h_i \oplus d_j) + b$$

Where:

h_i = head representation (from MLP)

d_j = dependent representation (from MLP)

U, W, b = learned parameters

$\oplus$ = vector concatenation

All Possible Arcs (Head → Dependent)

Below are **plausible arcs** with **sample weights** (e.g., predicted by a neural model). Higher weight = more likely correct.

Head	Dependent	Relation	Weight (score)
ROOT	plays	root	1.00
plays	boy	nsubj	0.95
plays	soccer	dobj	0.92
boy	The	det	0.90
plays	.	punct	0.85
ROOT	boy	nsubj	0.50
ROOT	soccer	dobj	0.30
soccer	The	??	0.10
plays	The	det	0.20

Build Directed Graph

All possible arcs are edges with weights. We must now find a **maximum spanning tree (MST)** rooted at ROOT, covering all words with one incoming arc each (except ROOT).

Select Arcs for MST (Best Dependency Tree)

Using Chu-Liu/Edmonds' algorithm, we select the highest-scoring, valid tree:

Selected Arcs:

ROOT → **plays** (1.00) → root

plays → **boy** (0.95) → nsubj

boy → **The** (0.90) → det

plays → **soccer** (0.92) → dobj

plays → **.** (0.85) → punct

Final Dependency Tree

Head	Dependent	Relation
ROOT	plays	root

plays	boy	nsubj
boy	The	det
plays	soccer	dobj
plays	.	punct